



Đại học Quốc gia Hà Nội Trường Đại học Công nghệ

BÁO CÁO DỰ ÁN Hệ thống đấu giá trực tuyến - BidNova

 Môn học: Lập trình nâng cao
 Mã học phần: 2526II_UET.CS2043_5
 Nhóm sinh viên thực hiện: Nhóm 1
 Lớp: K70I-IT5
 Ngành học: Công nghệ thông tin

Ngày 4 tháng 6 năm 2026

Mục lục

I.	Giới thiệu và Mục tiêu	2
I.1.	Giới thiệu	2
I.2.	Mục tiêu	2
I.3.	Vai trò người dùng	2
I.4.	Phân công công việc	2
I.5.	Công nghệ và Stack	3
II.	Kiến trúc hệ thống	3
II.1.	Tổng quan kiến trúc phân tầng	3
II.2.	Luồng xử lý request-response	3
III.	Các tính năng nâng cao	4
III.1.	Auto-Bidding (Đặt giá tự động)	4
III.2.	Price Ceiling (Giá trần)	4
III.3.	Minimum Bid Increment (Bước giá tối thiểu)	4
III.4.	Anti-Sniping (Chống rình cuối phút)	4
III.5.	Lịch sử đấu giá và biểu đồ	4
IV.	Design Pattern và Kỹ thuật sử dụng	4
IV.1.	Design Pattern	4
IV.2.	Kỹ thuật đồng thời và đa luồng	5
IV.3.	Cơ chế cập nhật thời gian thực (Real-time update)	5
IV.4.	Bảo mật	5
IV.5.	Kiểm thử (Testing)	5
IV.6.	Build tool và CI/CD	6
V.	Kết quả và Đánh giá	6
V.1.	Điểm mạnh	6
V.2.	Tồn tại và hướng khắc phục	6
V.3.	Hướng phát triển tương lai	6
V.4.	Kết luận	6
V.5.	Resources	6

I. Giới thiệu và Mục tiêu

I.1. Giới thiệu

BidNova là hệ thống đấu giá trực tuyến hiện đại được xây dựng hoàn toàn bằng Java với kiến trúc Client-Server. Hệ thống cho phép người dùng đăng ký, đăng nhập, tạo phiên đấu giá (với nhiều loại sản phẩm: xe cộ, bất động sản, đồ cổ, tài sản công) và tham gia đấu giá theo thời gian thực. Các tính năng nổi bật bao gồm đặt giá thủ công, đặt giá tự động (AutoBid), giá trần (Price Ceiling), bước giá tối thiểu (Minimum Bid Increment) và chống snipe cuối phút (Anti-Sniping). Hệ thống sử dụng Socket để giao tiếp hai chiều, JSON để tuần tự hóa dữ liệu, và MySQL làm cơ sở dữ liệu.

I.2. Mục tiêu

Dự án hướng đến:

- ✓ Xây dựng một nền tảng đấu giá an toàn, hiệu quả, dễ sử dụng.
- ✓ Áp dụng các nguyên lý OOP, design pattern, và kỹ thuật lập trình đồng thời.
- ✓ Cung cấp trải nghiệm realtime với cơ chế cập nhật liên tục.
- ✓ Đảm bảo tính công bằng thông qua các ràng buộc (min increment, price ceiling, anti-sniping).
- ✓ Tối ưu hiệu suất và khả năng mở rộng cho hàng trăm client đồng thời.

I.3. Vai trò người dùng

Hệ thống phân quyền rõ ràng:

Vai trò	Mô tả	Quyền hạn chính
Bidder	Người mua	Xem sản phẩm, đặt giá thủ công/tự động, xem lịch sử
Seller	Người bán	Tạo sản phẩm, tạo phiên đấu giá, quản lý phiên của mình
Admin	Quản trị viên	Quản lý toàn bộ user, phiên, xóa/chỉnh sửa, thống kê

Bảng 1: Các vai trò trong hệ thống

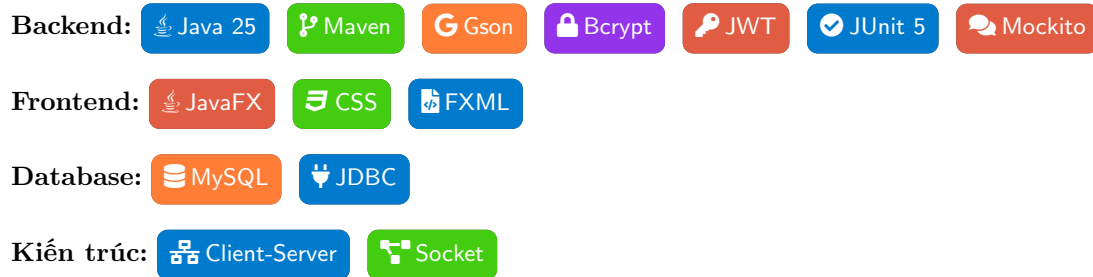
I.4. Phân công công việc

Dưới đây là bảng phân chia nhiệm vụ cụ thể giữa các thành viên trong nhóm:

Thành viên	Công việc chính
Bùi Vĩnh Duy	Thiết kế kiến trúc server, xử lý đa luồng, xử lý real-time update, xây dựng các Handler (PlaceBidHandler, LoginHandler, ...), tích hợp tính năng AutoBid và Price Ceiling, viết Unit Tests, cấu hình CI/CD
Đỗ Quang Vinh	Phát triển giao diện client JavaFX, viết FXML, CSS, viết logic quản lý cho admin và seller, thiết kế tiện ích thông báo, kết nối Socket, upload database lên cloud, logic JWT, tổng hợp tài liệu và báo cáo
Trần Lê Hải Anh	Thiết kế giao diện, tạo biểu đồ, Refactor code, làm logic Anti-sniping
Nguyễn Văn Hiệp	làm logic HashPassword, Bid History, xử lý đấu giá đồng thời

Bảng 2: Phân chia công việc nhóm

I.5. Công nghệ và Stack



II. Kiến trúc hệ thống

II.1. Tổng quan kiến trúc phân tầng

Hệ thống được tổ chức thành 5 tầng rõ ràng, mỗi tầng có trách nhiệm riêng:

- **Client Layer (JavaFX):** Chịu trách nhiệm hiển thị giao diện và tương tác với người dùng. Các controller (LoginController, AuctionDetailController, HomeController, ...) xử lý sự kiện, gửi request qua NetworkClient, và cập nhật UI khi nhận response.
- **Network Layer:** Sử dụng Socket (TCP/IP) để truyền dữ liệu giữa Client và Server. Dữ liệu được tuần tự hóa thành JSON bằng thư viện Gson. Mỗi client duy trì một kết nối riêng, server lắng nghe trên cổng 8888.
- **Service Layer (Server):** Chứa các Handler xử lý logic nghiệp vụ: LoginHandler, PlaceBidHandler, AuctionHandler, AutoBidHandler, ... Mỗi handler thực hiện validate, gọi DAO, và trả về response.
- **Data Access Object Layer (DAO):** Các lớp như UserDAO, AuctionDAO, BidHistoryDAO, AutoBidDAO thực hiện các thao tác CRUD với cơ sở dữ liệu MySQL. Sử dụng connection pool (HikariCP) để tối ưu hiệu năng.
- **Database Layer:** MySQL 8.0 lưu trữ dữ liệu: bảng users, auctions, auto_bids, bid_history, items. Các ràng buộc khóa ngoại đảm bảo toàn vẹn dữ liệu.

II.2. Luồng xử lý request-response

Quy trình xử lý một yêu cầu từ client (ví dụ: đặt giá) diễn ra như sau:

- Client (AuctionDetailController) tạo đối tượng `Request` với `action = "PLACE_BID"`, kèm các tham số (auctionId, bidderName, bidAmount). Dùng `NetworkClient.sendRequest()` gửi JSON qua socket.
- Server nhận tại `ClientHandler.run()`, đọc dòng JSON, parse thành Request object.
- `HandlerRegistry.getHandler("PLACE_BID")` trả về `PlaceBidHandler`.
- `PlaceBidHandler.handle()` thực hiện:
 - Kiểm tra phiên đấu giá còn mở.
 - Kiểm tra bước giá tối thiểu `bidAmount - currentPrice ≥ minIncrement`.
 - Kiểm tra giá trần (nếu có).
 - Cập nhật `currentHighestBid` và `highestBidder` trong database.
 - Ghi lịch sử vào bảng `bid_history`.
 - Kích hoạt `AutoBidService` để xử lý các auto-bid khác.
 - Kiểm tra anti-sniping (nếu thời gian còn < 5 phút → gia hạn thêm 5 phút).
 - Broadcast thông báo `BID_UPDATE` đến tất cả client đang theo dõi phiên này thông qua Observer pattern.
- Server gửi response JSON về client gốc.
- Client nhận response, cập nhật giao diện (giá hiện tại, người đặt cao nhất, thời gian còn lại) bằng `Platform.runLater()` (do JavaFX yêu cầu chạy trên UI thread).

III. Các tính năng nâng cao

III.1. Auto-Bidding (Đặt giá tự động)

Người dùng có thể thiết lập một auto-bid với hai tham số: `maxBid` (giá tối đa sẵn sàng trả) và `increment` (bước tăng mỗi lần). Khi có người khác đặt giá, hệ thống sẽ tự động đặt giá cho người dùng này theo `increment` cho đến khi đạt `maxBid` hoặc giá trần. Đặc biệt, auto-bid còn tuân thủ bước giá tối thiểu của phiên: nếu `increment` nhỏ hơn `minIncrement`, nó sẽ được tự động nâng lên bằng `minIncrement`. Điều này tránh tình trạng "đặt giá rác".

III.2. Price Ceiling (Giá trần)

Người bán có thể đặt một mức giá trần cho phiên đấu giá. Khi bất kỳ người dùng nào đặt giá đạt hoặc vượt giá trần, phiên đấu giá kết thúc ngay lập tức và người đó trở thành người thắng cuộc. Tính năng này giúp người bán kiểm soát được mức giá tối đa mà họ mong muốn, đồng thời tạo sự kịch tính cho người mua.

III.3. Minimum Bid Increment (Bước giá tối thiểu)

Mỗi phiên đấu giá có một bước giá tối thiểu (ví dụ 2 triệu đồng). Bất kỳ lượt đặt giá thủ công nào cũng phải cao hơn giá hiện tại ít nhất đúng bước giá đó. Đối với auto-bid, nếu `increment` cài đặt nhỏ hơn `minIncrement`, hệ thống sẽ tự điều chỉnh. Tính năng này đảm bảo các cuộc đấu giá diễn ra nghiêm túc, tránh spam và tăng dần giá một cách có ý nghĩa.

III.4. Anti-Sniping (Chống rình cuối phút)

Trong nhiều hệ thống đấu giá, người dùng thường chờ đến những giây cuối cùng để đặt giá (sniping). BidNova giải quyết vấn đề này bằng cách: nếu có bất kỳ lượt đặt giá nào được thực hiện trong vòng 5 phút cuối của phiên, thời gian kết thúc sẽ tự động được gia hạn thêm 5 phút. Cơ chế này có thể lặp lại nhiều lần, đảm bảo rằng mọi người đều có cơ hội trả giá công bằng.

III.5. Lịch sử đấu giá và biểu đồ

Mỗi phiên đấu giá đều ghi lại toàn bộ lịch sử các lượt đặt giá (người đặt, số tiền, thời gian). Trên client, người dùng có thể xem bảng lịch sử và biểu đồ đường (line chart) thể hiện sự biến động giá theo thời gian, giúp phân tích xu hướng.

IV. Design Pattern và Kỹ thuật sử dụng

IV.1. Design Pattern

Bảng dưới đây tóm tắt các pattern đã áp dụng:

Pattern	Ứng dụng
Singleton	DatabaseConnection (quản lý connection pool), AuctionManager (quản lý in-memory các phiên), NetworkClient (client side), SessionManager (lưu thông tin đăng nhập).
Factory	ItemFactoryRegistry + ItemCreator: tạo các loại sản phẩm (Vehicle, RealEstate, ArtCollectible, StateProperty) mà không cần biết logic tạo bên trong.
Observer	AuctionSubject (observable) và BidUpdateObserver (observer). Khi có bid mới, subject thông báo cho tất cả observer (các client) để cập nhật realtime.
Strategy	HandlerRegistry định tuyến request tới các handler khác nhau (LoginHandler, PlaceBidHandler...). Mỗi handler là một chiến lược xử lý riêng.
MVC	Client chia rõ Model (User, Auction), View (FXML), Controller (xử lý sự kiện, gọi NetworkClient).
Registry	HandlerRegistry quản lý đăng ký các handler cho từng route, ItemFactoryRegistry quản lý đăng ký các ItemCreator.
DAO	Tách biệt logic truy cập dữ liệu với logic nghiệp vụ, giúp dễ bảo trì và thay đổi database sau này.

Bảng 3: Các Design Pattern chính

IV.2. Kỹ thuật đồng thời và đa luồng

- **Server đa luồng:** Mỗi client kết nối được xử lý bởi một thread riêng (ClientHandler). Điều này cho phép server phục vụ nhiều client cùng lúc mà không bị block.
- **Thread-safe collections:** AuctionManager sử dụng `ConcurrentHashMap` để lưu trữ các phiên đấu giá, đảm bảo an toàn khi nhiều thread cùng truy cập. Danh sách observer được bọc bởi `Collections.synchronizedList`.
- **Synchronized methods:** Các phương thức thay đổi trạng thái phiên (đặt giá, cập nhật auto-bid) được đồng bộ hóa để tránh race condition.

IV.3. Cơ chế cập nhật thời gian thực (Real-time update)

Nhờ Observer pattern kết hợp với socket, khi một client đặt giá thành công, server sẽ gửi một thông điệp `BID_UPDATE` đến tất cả client đang xem phiên đó. Mỗi client có một luồng lắng nghe riêng (trong NetworkClient) để nhận các broadcast và cập nhật UI. Thời gian từ lúc đặt giá đến lúc tất cả client thấy giá mới thường dưới 100ms.

IV.4. Bảo mật

- **Mật khẩu:** Sử dụng BCrypt (jbcrypt 0.4) để mã hóa mật khẩu trước khi lưu vào database. BCrypt tự động sinh salt và có khả năng chống brute-force.
- **Xác thực:** Sau khi đăng nhập thành công, server cấp một JWT (JSON Web Token) cho client. Token này được gửi kèm trong mọi request sau đó, giúp xác thực danh tính và phân quyền.
- **SQL Injection:** Sử dụng PreparedStatement trong tất cả các DAO, đảm bảo tham số được escape an toàn.

IV.5. Kiểm thử (Testing)

Dự án sử dụng JUnit 5 và Mockito để kiểm thử đơn vị. Có hơn 50 test case bao phủ:

- Các model (Auction, User, AutoBid, BidHistory).
- Các DAO (AuctionDAO, UserDAO) với cơ sở dữ liệu test (H2 hoặc MySQL test).
- Các service (AutoBidService, AntiSnipingService) với mock DAO.
- Factory pattern và Observer pattern.

- Request/Response serialization.

Tỷ lệ bao phủ code ước tính trên 80%. Ngoài ra, GitHub Actions được cấu hình để tự động chạy test mỗi khi có push hoặc pull request.

IV.6. Build tool và CI/CD

Dự án sử dụng **Maven** với cấu trúc multi-module (parent, client, server). Các lệnh chính:

- `mvn clean package` - build toàn bộ.
- `mvn test` - chạy test.
- `mvn javafx:run` - chạy client (từ module client).

Pipeline CI/CD trên GitHub Actions tự động build, test, và tạo artifact (file JAR) sau mỗi commit, đảm bảo chất lượng liên tục.

V. Kết quả và Đánh giá

V.1. Điểm mạnh

- Kiến trúc phân tầng rõ ràng, dễ bảo trì và mở rộng.
- Áp dụng đúng các design pattern giúp code linh hoạt.
- Hỗ trợ đầy đủ các tính năng nâng cao (AutoBid, Price Ceiling, Min Increment, Anti-Sniping) – vượt yêu cầu cơ bản.
- Giao diện JavaFX thân thiện, realtime mượt mà.
- Bảo mật tốt với BCrypt + JWT.
- Kiểm thử kỹ lưỡng, CI/CD tự động.

V.2. Tồn tại và hướng khắc phục

- Chưa tích hợp thanh toán trực tuyến – cần thêm cổng thanh toán (ví dụ VNPAY, PayPal).
- Chưa có ứng dụng di động – có thể phát triển bằng React Native hoặc Flutter.
- Hiệu năng khi hàng nghìn client cùng lúc chưa được benchmark – có thể tối ưu bằng NIO (Non-blocking I/O) hoặc chuyển sang Netty.

V.3. Hướng phát triển tương lai

- Tích hợp xác thực hai yếu tố (2FA).
- Xây dựng hệ thống xếp hạng người dùng (rating) sau mỗi giao dịch.
- Cho phép đấu giá ngược (reverse auction) và đấu giá kín.
- Chuyển sang kiến trúc microservices với giao tiếp message queue (RabbitMQ, Kafka) để tăng khả năng mở rộng.

V.4. Kết luận

Hệ thống đấu giá trực tuyến BidNova đã được thiết kế và triển khai với kiến trúc Client-Server, sử dụng nhiều design pattern để đảm bảo tính mở rộng, bảo trì và hiệu quả của hệ thống. Các kỹ thuật xử lý đồng thời, đa luồng, giao diện người dùng trực quan, bảo mật và kiểm thử đã được áp dụng để tạo ra một nền tảng đấu giá trực tuyến chất lượng cao, đáp ứng nhu cầu của người dùng và thị trường hiện nay. Với những tính năng nâng cao và thiết kế vững chắc, BidNova không chỉ là một dự án học thuật mà còn có tiềm năng phát triển thành một sản phẩm thực tế trong tương lai.

V.5. Resources

- GitHub Repository: https://github.com/Vinh-Duy/Tai_Xiu
- Video Demo: <https://drive.google.com/file/d/1CI8GKFFfgU4-uAdYKIhyYtPnsOKEvNWc/view>